

COMO PROGRAMAR EM PYTHON

DO ZERO



Augusto Patsch
@imperio_do_programador

SUMÁRIO

INTRODUÇÃO.....	04
Por que aprender Python?.....	04
Aplicações em Python.....	05
INSTALAÇÃO PYTHON.....	06
Windows.....	06
Linux.....	08
MacOS.....	09
INSTALAÇÃO PYCHARM.....	10
Windows.....	10
Linux.....	15
MacOS.....	17
VARIÁVEIS.....	19
O que são variáveis?.....	19
Regras para declarar variáveis.....	19
Tipagem dinâmica.....	19
Tipos de variáveis	20
Input()	21
STRINGS.....	22
O que é string?.....	22
Concatenação.....	22
Manipulação.....	22
Fatiamento.....	26
NÚMEROS.....	27
Operadores aritméticos.....	27
Operadores de comparações.....	27
Operadores lógicos.....	28

SUMÁRIO

COLEÇÕES.....	29
Listas.....	29
Tuplas.....	33
Dicionários.....	34
ESTRUTURA DE DECISÃO.....	35
If.....	35
If Else.....	36
If Elif Else.....	37
ESTRUTURA DE REPETIÇÃO.....	38
While.....	38
For.....	39
FUNÇÕES.....	40
O que é função?.....	40
Sintaxe.....	40
Parâmetros e Argumentos.....	40
Escopo de variáveis.....	41
Retorno de valores.....	41
BIBLIOGRAFIA.....	42

INTRODUÇÃO

POR QUE PYTHON?

FACILIDADE

A linguagem Python é uma das mais fáceis de aprender. Se você é iniciante no mundo da programação, não terá grandes dificuldades, basta ter uma noção básica da língua inglesa e irá aprender tranquilamente. É devido a sua simplicidade, que cursos de programação para iniciantes comumente utilizam a linguagem Python.

É PORTÁTIL

Dizer que python é uma linguagem portátil, significa dizer que conseguimos rodar o mesmo programa em diversos sistemas operacionais sem precisar mudar o código.

PRATICIDADE

Python é uma linguagem muito mais prática que as demais, um programa feito em python geralmente possui muito menos linhas de códigos necessárias para executar uma tarefa do que C/C++ e Java, por exemplo. Vamos pegar um exemplo de um programa que sua tarefa é abrir um arquivo csv.

Em Java:

```
1. package stack;
2.
3. import java.io.BufferedReader;
4. import java.io.FileNotFoundException;
5. import java.io.FileReader;
6. import java.io.IOException;
7.
8. public class LeiaCSV {
9.
10.     public static void main(String[] args) {
11.
12.         LeiaCSV obj = new LeiaCSV();
13.         obj.run();
14.     }
15.
16.     public void run() {
17.
18.         String arquivoCSV = "arquivo.csv";
19.         BufferedReader br = null;
20.         String linha = "";
21.         String csvDivisor = ",";
22.         try {
23.
24.             br = new BufferedReader(new FileReader(arquivoCSV));
25.             while ((linha = br.readLine()) != null) {
26.
27.                 String[] pais = linha.split(csvDivisor);
28.
29.                 System.out.println("País [code= " + pais[pais.length-2]
30.                                     + " , name=" + pais[pais.length-1] +
31.                                     "]\n");
32.             }
33.
34.         }
```

```
34.
35.         } catch (FileNotFoundException e) {
36.             e.printStackTrace();
37.         } catch (IOException e) {
38.             e.printStackTrace();
39.         } finally {
40.             if (br != null) {
41.                 try {
42.                     br.close();
43.                 } catch (IOException e) {
44.                     e.printStackTrace();
45.                 }
46.             }
47.         }
48.     }
49.
50. }
```

Em Python (Biblioteca *Pandas*):

```
1. import pandas as pd
2. df = pd.read_csv('planilha.csv')
```

É GRÁTIS

Para programar em Python você não precisa de nenhuma licença, ela roda em qualquer computador ou notebook. Outros recursos como módulos, bibliotecas e IDEs são totalmente gratuitos.

GRANDES EMPRESAS USAM

Python é uma linguagem muito usada em diversas empresas do mundo, inclusive por grandes empresas como Google, Instagram, Amazon, Spotify, Facebook, Dropbox, Nasa, Eletronic Arts, e Disney

BIBLIOTECAS

Python tem um número gigante de bibliotecas que irão te auxiliar na hora de programar. Com as bibliotecas, você pode utilizar códigos já feitos por outras pessoas sem ter que inventar tudo do zero.

APLICAÇÕES PYTHON

DATA SCIENCE

DataScience é campo cujo principal objetivo é estudar a análise de dados extraídos de diversas fontes para obter informações e posteriormente elaborar previsões. As bibliotecas mais usadas é Pandas, NumPy e SciPy.

MACHINE LEARNING

Também conhecido como Aprendizado de Máquinas, a Machine Learning é um ramo da Inteligência Artificial. Ela automatiza a construção de modelos analíticos a partir de uma análise de dados para que a máquina possa identificar padrões e tomar decisões com a mínima interferência humana possível.

DESENVOLVIMENTO WEB

Através de dois frameworks muito utilizados, Django e Flask, é possível criar aplicações e sites que são acessados diretamente do navegador, em computadores, celulares e tablets.

SCRIPTING

São programas curtos que automatizam tarefas do dia-a-dia que poderiam ser feitas pelos próprios humanos, como enviar emails e enviar mensagens automáticas no WhatsApp.

JOGOS

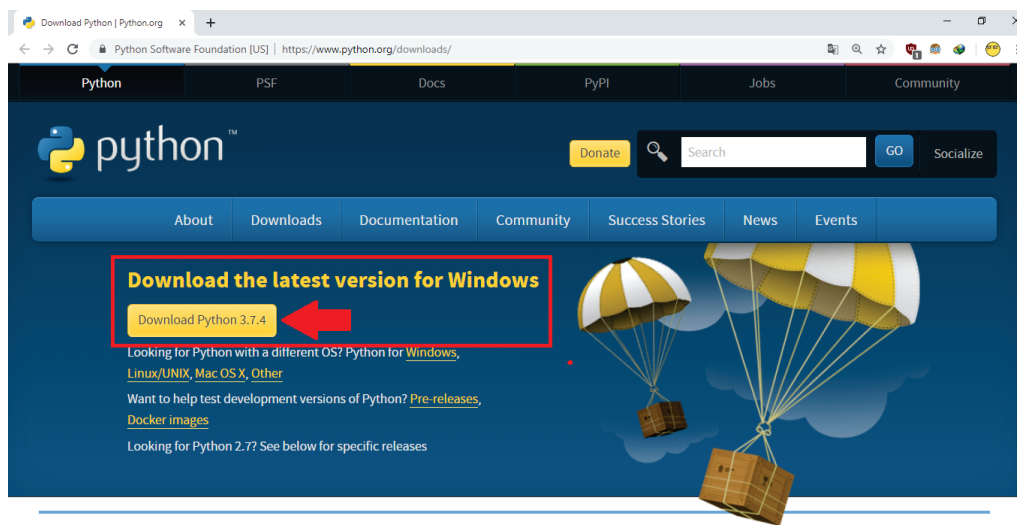
Também é possível a criação de jogos simples, como FlappyBird, jogo da cobrinha, jogo da memória, entre outros, através da biblioteca PyGame.

Estas são apenas algumas das principais aplicações da linguagem Python. Com Python, é possível fazer praticamente tudo que você quiser!

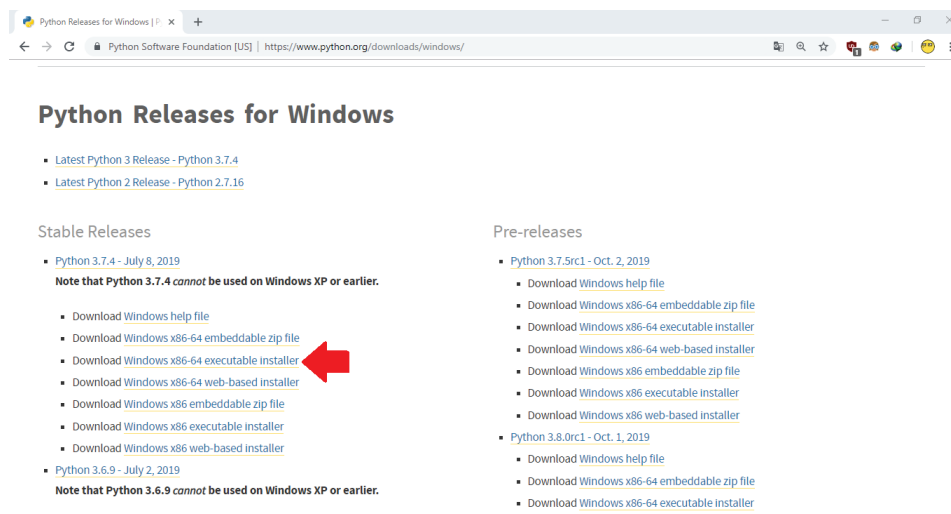
INSTALAÇÃO PYTHON

WINDOWS

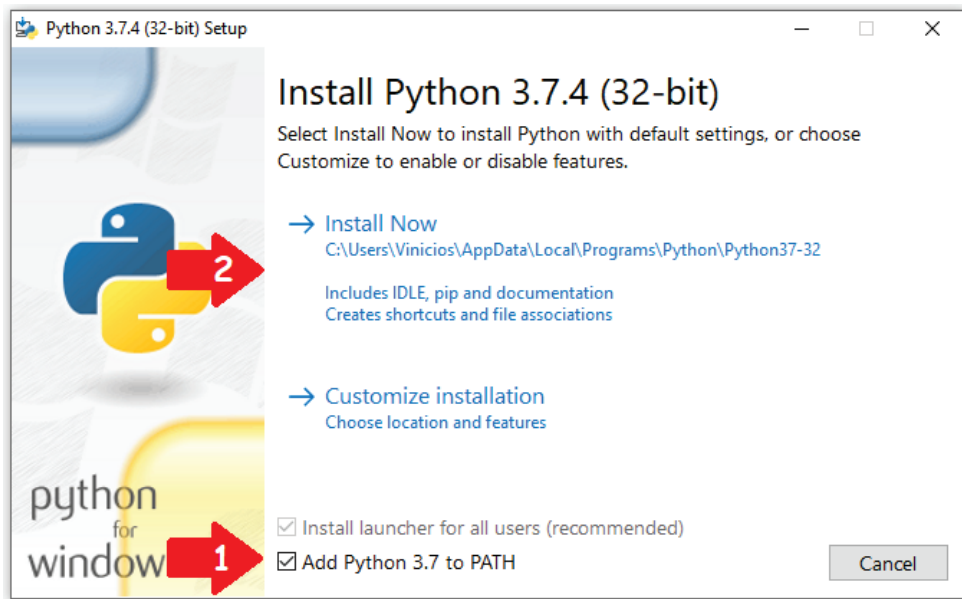
Para instalar o Python no Windows, você precisa baixar o instalador, que você encontra no o site oficial (www.python.org/downloads/). Clique em download.



Por padrão, será feito o download do Python 3 para sistemas de 32 bits. Para o instalador de 64 bits, clique em windows e selecione o instalador de 64 bits apropriado, como mostrado abaixo.



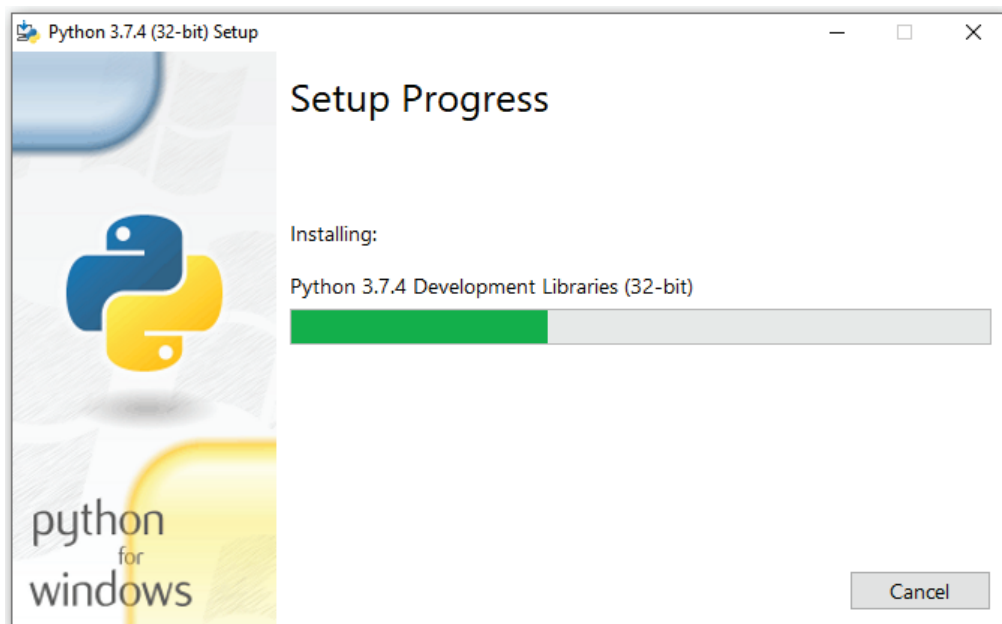
Faça o download do instalador executável do Windows (32 ou 64 bits) e clique duas vezes nele para iniciar o assistente de instalação do python, como mostrado abaixo.



A instalação é bem fácil:

1. Ative a opção "Add Python to PATH"
2. Clique em "Install Now"

Aguarde a instalação ser finalizada



Se tudo ocorrer bem, a próxima tela será mostrada. Clique em "Close".

Instalação por Gerenciadores de Pacotes

O apt-get (Debian, Ubuntu) e yum (RedHat, CentOS) são os gerenciadores de pacotes mais comuns. Caso você não utilize um algum destes gerenciador de pacotes, acesse a página de downloads do Python.

Apt-get

Para instalar o Python 3, digite em um terminal:

```
$ sudo apt-get install python3
```

Para instalar o gerenciador de pacotes pip, digite em um terminal:

```
$ sudo apt-get install python3-pip
```

Yum

Para instalar o Python 3, digite em um terminal:

```
$ sudo yum install python3
```

Para instalar o gerenciador de pacotes pip, digite em um terminal:

```
$ yum -y install python3-pip
```

MacOS

Se você possui a versão macOS 10.2 ou superior, é provável que você já tenha alguma versão do python instalada.

Para conferir, digite no terminal:

```
$ which python
```

ou

```
$ which python3
```

Se retornar algo como `/usr/bin/python` significa que você já tem o python instalado.

INSTALAÇÃO

Antes de instalar o Python, instale o Xcode (App Store) do pacote para desenvolvimento em linha de comando no macOS, command line tools e dos gerenciadores de pacotes pip e homebrew.

Para instalar o command line tools, digite em um terminal:

```
$ xcode-select --install
```

Para instalar o pip, digite em um terminal:

```
$ sudo easy_install pip
```

Para atualizar o pip, digite em um terminal:

```
$ sudo pip install --upgrade pip
```

Para instalar o homebrew, digite em um terminal:

```
$ ruby -e "$(curl -fsSL https://raw.githubusercontent.com/mxcl/homebrew/go)"
```

Para instalar o Python 2, digite em um terminal:

```
$ brew install python
```

Para instalar o Python 3, digite em um terminal:

```
$ brew install python3
```

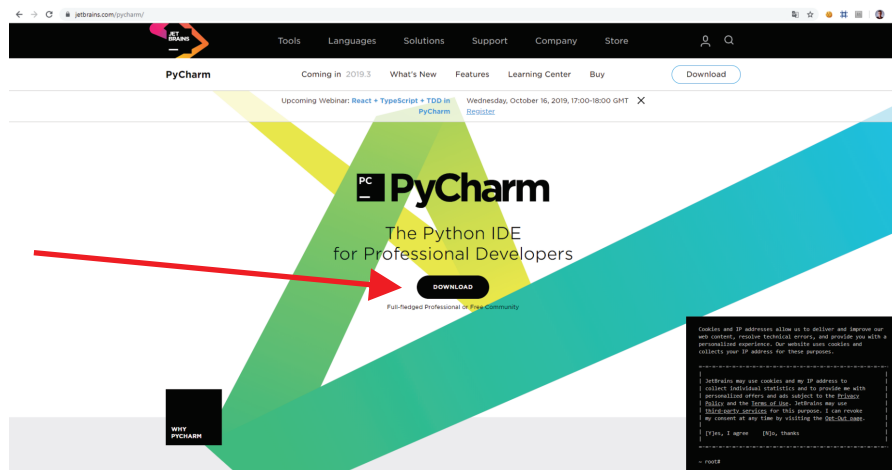
INSTALAÇÃO PYCHARM

Com o python já instalado, agora é preciso ter um editor de texto para que possamos escrever nossos códigos.

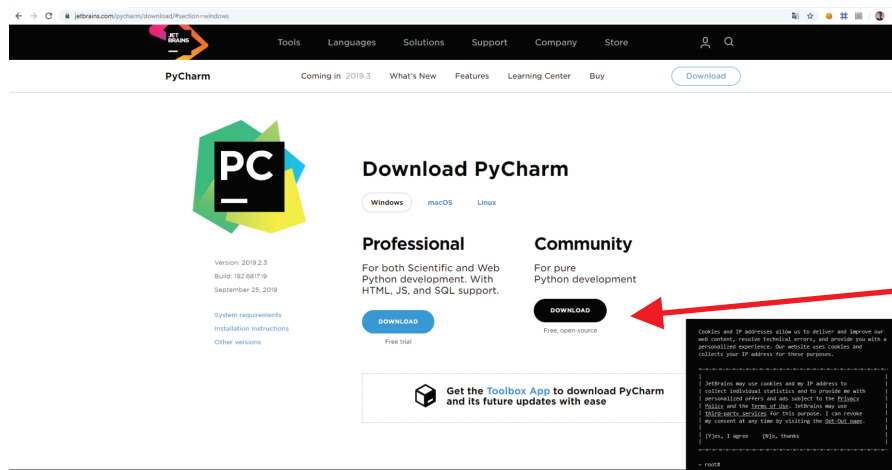
Para isso, iremos utilizar o PyCharm, um dos IDE's mais utilizados para progamar em python.

WINDOWS

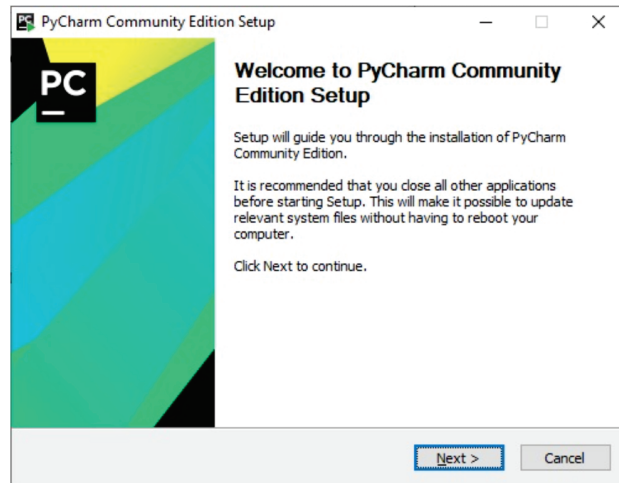
Para fazer a instalação do PyCharm, entre no site da IDE (www.jetbrains.com/pycharm/) e clique no botão *download*



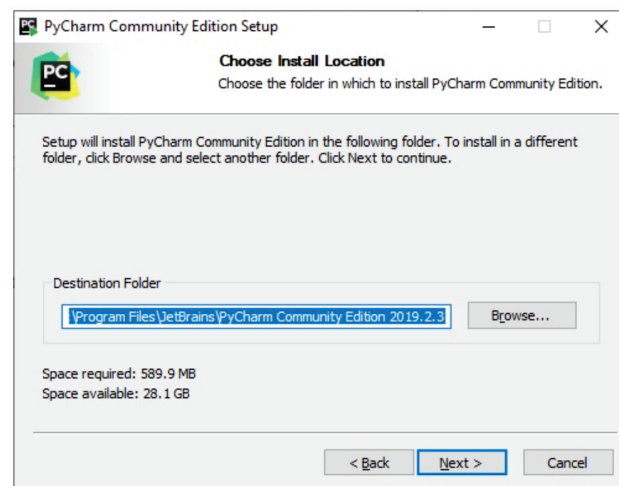
Escolha a opção **community** e clique em **download**



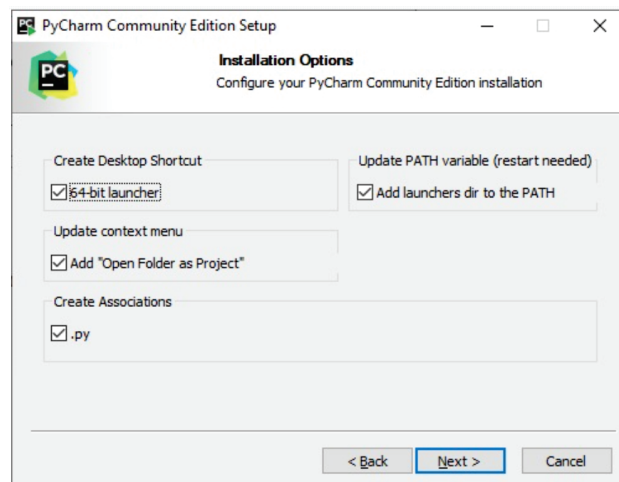
Welcome to PyCharm: Clique em next



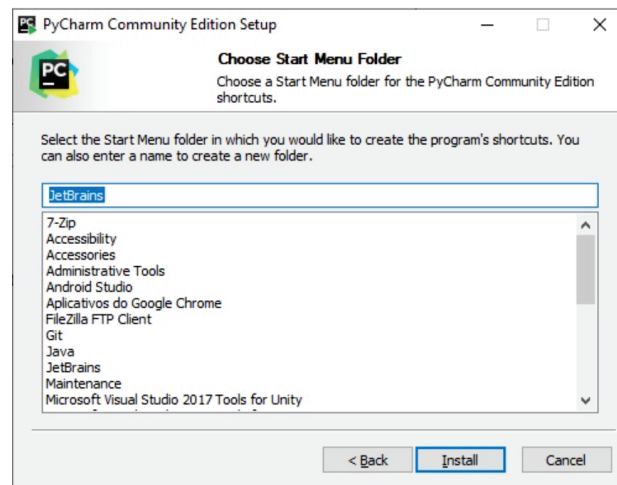
Choose install Location: mantenha o destino e clique em next



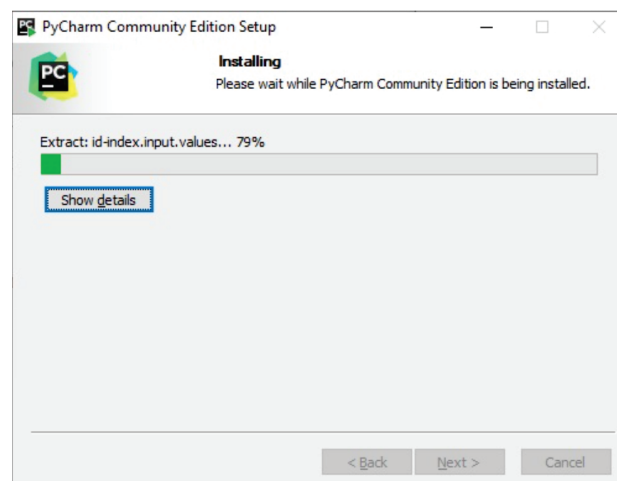
Install Options: marque todas as opções apresentadas e clique em next



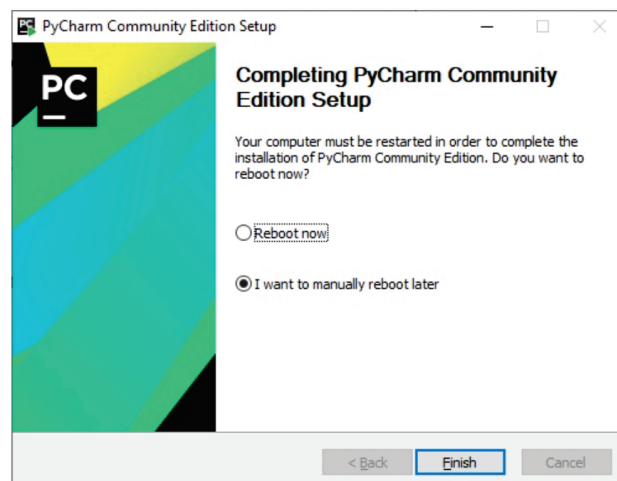
Choose Start menu folder: não selecionaremos nenhuma opção, apenas clicamos em Install



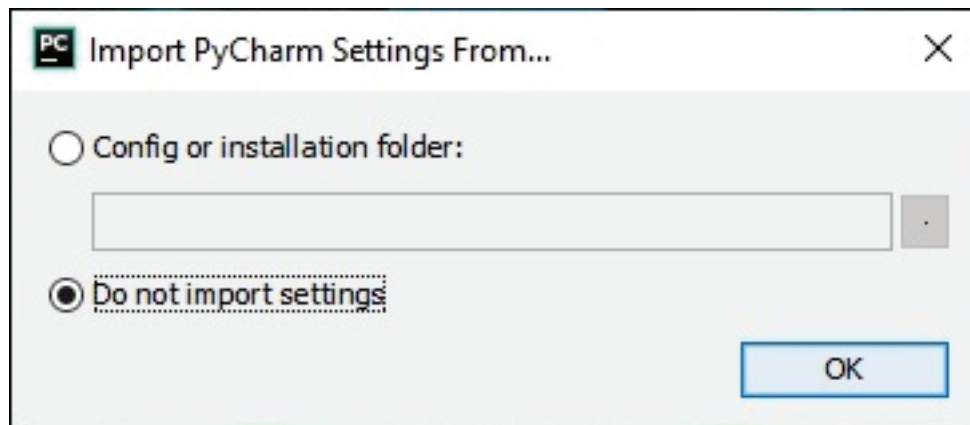
Na próxima tela veremos o progresso da instalação



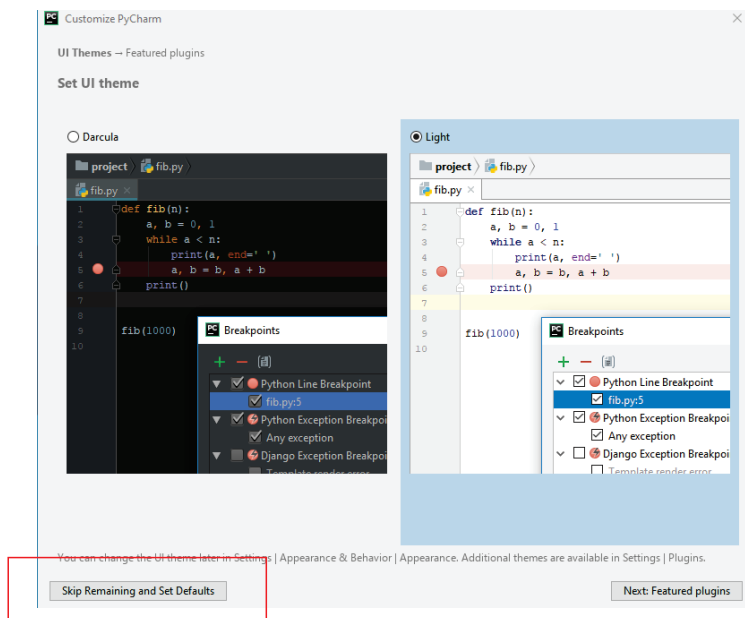
Ao final, surgirá a opção de rebotar a máquina, contudo, vamos selecionar a opção "I want to manually reboot later" e em seguida clicamos em Finish.



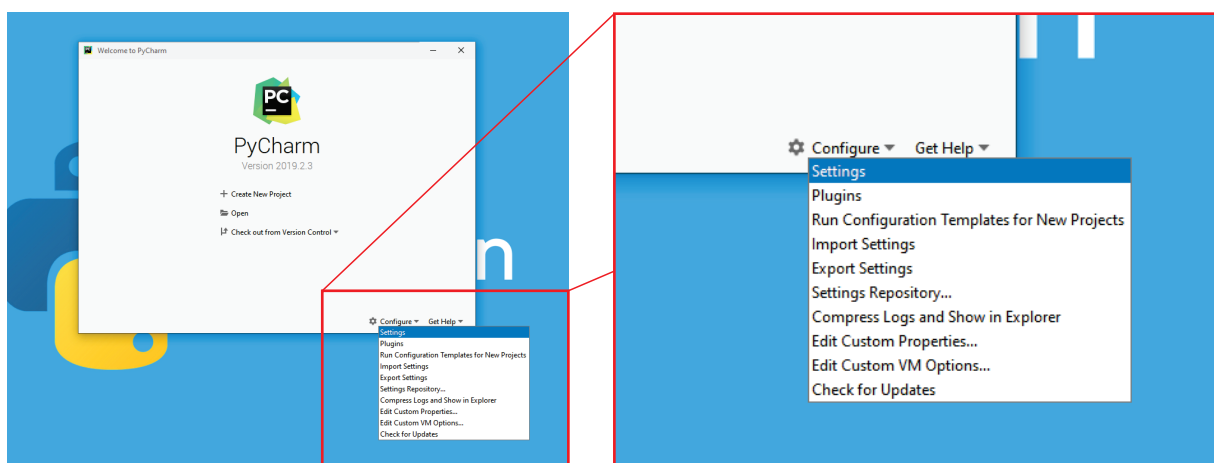
Abra o PyCharm e selecione a opção “Do not import settings” e clique em OK



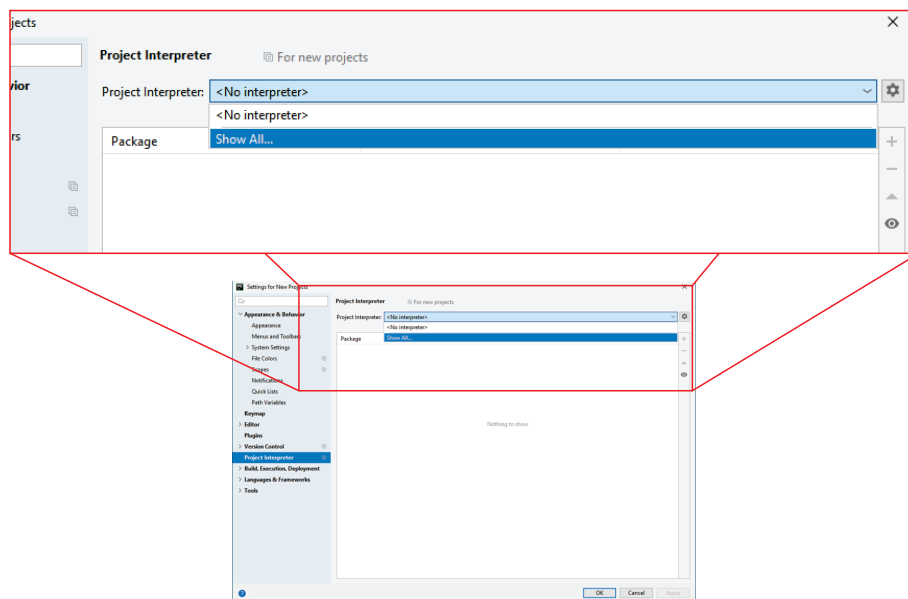
Escolha a opção que mais lhe agrada e clique em “Skip Remaining and Set Default”



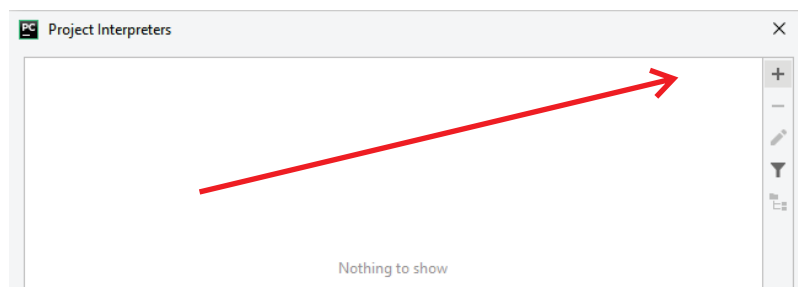
Precisamos informar ao programa onde o Python está instalado. Para isso, acesse as configurações e settings



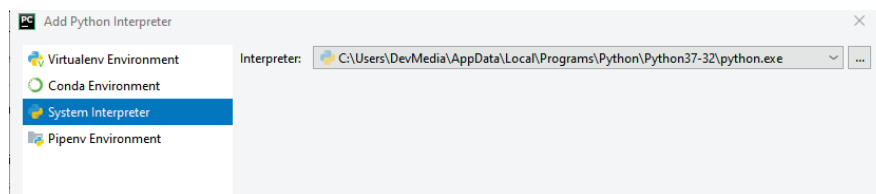
Em Project Interpreter, clique na opção Show All



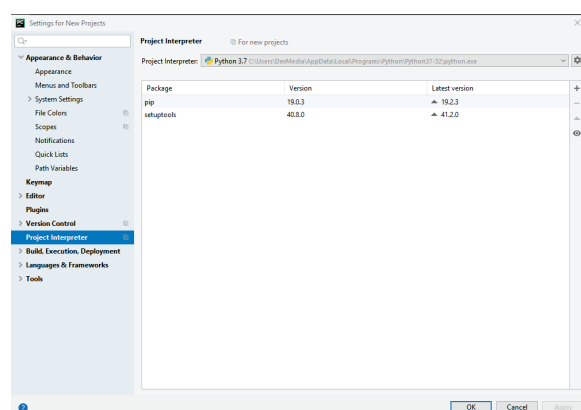
Clique no '+' no canto direito



Escolha a opção System Interpreter e clique em OK



Agora, só clicar em OK novamente e o PyCharm está pronto para ser usado!



LINUX

VIA SNAP

Abra um terminal

Instale o IDE PyCharm **community** usando esse comando:

```
sudo snap install pycharm-community --classic
```

Instale o IDE PyCharm **professional** usando esse comando:

```
sudo snap install pycharm-professional --classic
```

Atualizar o snap da IDE PyCharm (mais tarde se necessário), use esse comando:

```
sudo snap refresh pycharm-community --classic
```

ou

```
sudo snap refresh pycharm-community pycharm-professional
```

para desinstalar o PyCharm, use esse comando:

```
sudo snap remove pycharm-community pycharm-professional
```

UBUNTU 16.04, 14.04, LINUX MINT 18,17 e seus derivados

Abra um terminal

Adicione o repositório do aplicativo com o seguinte comando:

```
echo "deb http://ppa.launchpad.net/viktor-krivak/pycharm/ubuntu zesty main" |  
sudo tee /etc/apt/sources.list.d/pycharm-ide.list'
```

Baixe e adicione a chave do repositório GetDeb com o comando:

```
wget -q -O - http://archive.getdeb.net/getdeb-archive.key | sudo apt-key add -
```

Atualize o Apt com o comando

```
sudo apt-get update
```

Para instalar a IDE PyCharm **community** use este comando:

```
sudo apt-get install pycharm
```

Para instalar a IDE PyCharm **Professional**, use este comando;

```
sudo apt install pycharm-professional
```

Para desinstalar, use o comando:

```
sudo apt remove --autoremove pycharm pycharm-professional
```

UBUNTU 17.04, 16.10 e seus derivados

Abra um terminal

Para adicionar o repositório do programa use o comando:

```
sudo add-apt-repository ppa:viktor-krivak/pycharm
```

Para atualizar o gerenciador de pacotes, use o comando:

```
sudo apt-get update
```

Para instalar a IDE PyCharm **community**, use o comando:

```
sudo apt-get install pycharm
```

Para instalar a IDE PyCharm **professional**, use o comando:

```
sudo apt install pycharm-professional
```

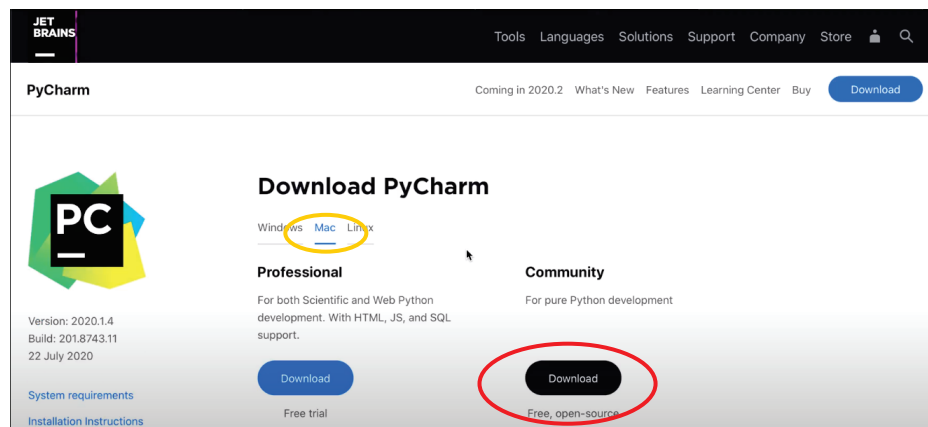
Para desinstalar, use o comando:

```
sudo apt remove --autoremove pycharm pycharm-professional
```

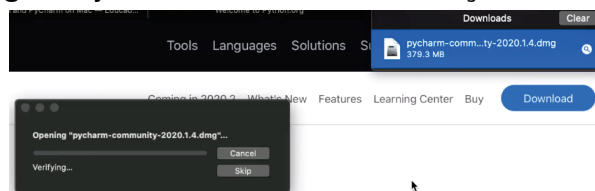
MacOS X

O Python está instalado, mas agora vamos exigir que um Ambiente de Desenvolvimento Integrado, também conhecido como IDE, seja instalado em seu computador para permitir que você codifique.

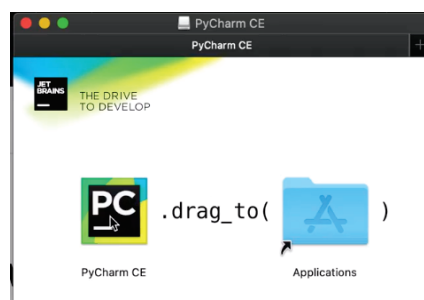
Vá para <https://www.jetbrains.com/pycharm/download/#section=mac>. Isso exibirá a versão mais recente do PyCharm. Abaixo da comunidade, baixe a versão gratuita e de código aberto do PyCharm.



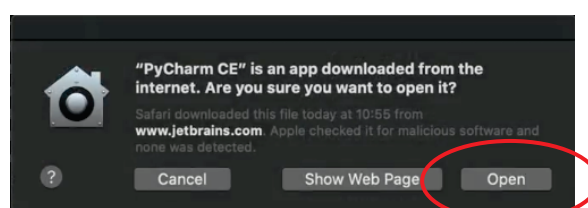
Clique no arquivo .dmg. O PyCharm deve então começar a se instalar.



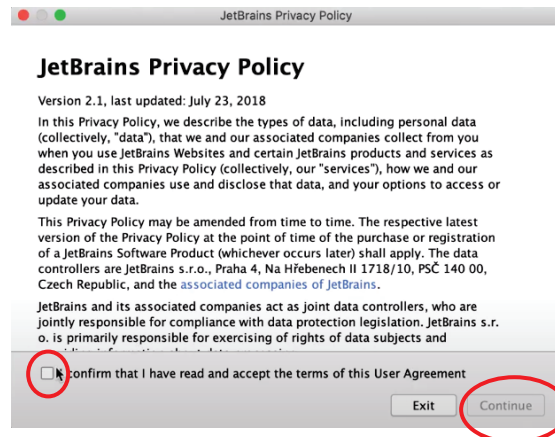
Arraste e solte o aplicativo na pasta Aplicativos.



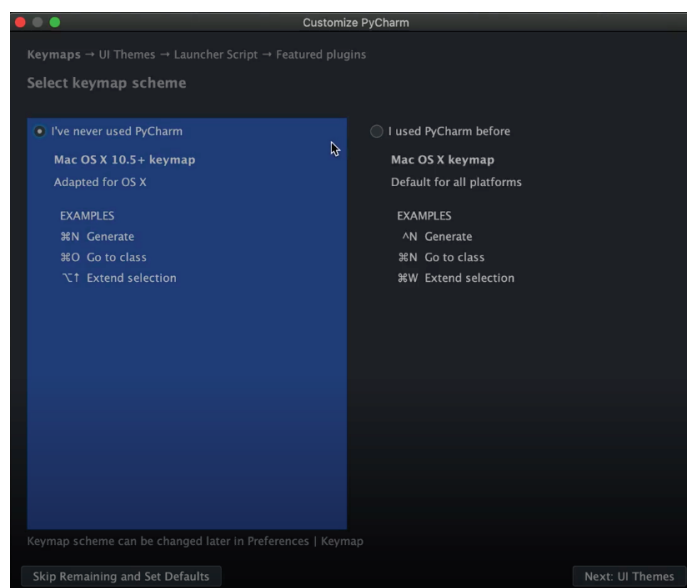
Selecione Abrir ao tentar executar o aplicativo pela primeira vez.



Agora iremos testar se PyCharm funciona executando o programa. Você será solicitado a ler a Política de Privacidade da JetBrains. Depois de ler, confirme que leu o Acordo do Usuário.



Agora você pode escolher entre diferentes opções para a instalação do Pycharm. Selecione “Nunca usei o Pycharm” para otimizar sua experiência como iniciante.



VARIÁVEIS

O QUE SÃO VARIÁVEIS?

Na programação, uma variável é um local de armazenamento (identificado por um endereço de memória) emparelhado com um nome simbólico associado, que contém alguma quantidade conhecida ou desconhecida de informação referida como um valor.

REGRAS PARA DECLARAR VARIÁVEIS

1. Iniciar o nome da variável com uma LETRA ou UNDERLINE.

- ✓ Exemplo = 10
- ✓ _Exemplo = 10
- ✗ 0Exemplo = 10
- ✗ #Exemplo = 10

2. O restante da variável pode conter letras, números e underline.

- ✓ Exemplo_com_0 = 10

3. Python é CaseSensitive, o que quer dizer que ele diferencia letras minúsculas e maiúsculas.

NUMERO = 10

numero = 5

NUMERO $\neq$ numero

4. Não pode utilizar as palavras reservadas do python como nome das variáveis.

'False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield'

TIPAGEM DINÂMICA

Em python, você não precisa especificar qual o tipo da variável antes de atribuir a ela um valor, como em C e Java, por exemplo. Aqui, o próprio interpretador infere o tipo de dado que uma variável recebe

TIPOS DE VARIÁVEIS

INTEIRO - é um número sem casa decimais, exemplo: 1, 3, 10, 10050.

```
a = 10  
print(type(a))
```

```
<class 'int'>
```

FLOAT - é um número com casa decimais, exemplo: 1.50, 3.55, 5.00.

```
b = 3.1415  
print(type(b))
```

```
<class 'float'>
```

STRING - é um ou mais caracteres, deve ser atribuído à variável com aspas simples ou duplas

```
c = "Olá Mundo"  
print(type(c))
```

```
<class 'str'>
```

BOOLEANO - é verdadeiro (True) ou falso (False)

```
d = True  
print(type(d))  
e = False  
print(type(e))
```

```
<class 'bool'>  
<class 'bool'>
```

print()

É uma função usada para imprimir os argumentos em um terminal

type()

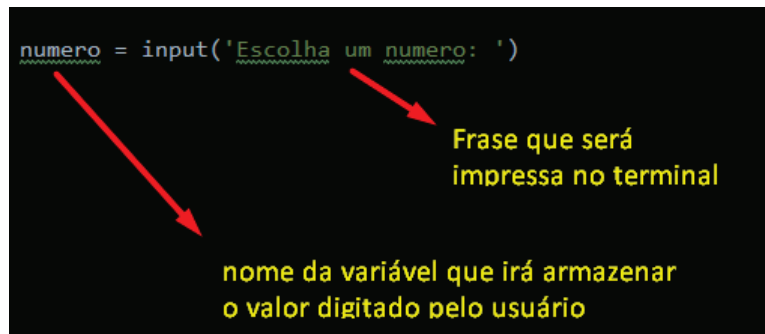
É uma função usada para identificar o tipo de uma variável

INPUT()

Input() é a função responsável pela entrada de dados no programa. Toda vez que o input for chamado será impresso no terminal a string passada como parâmetro e depois solicitar um valor para o usuário.

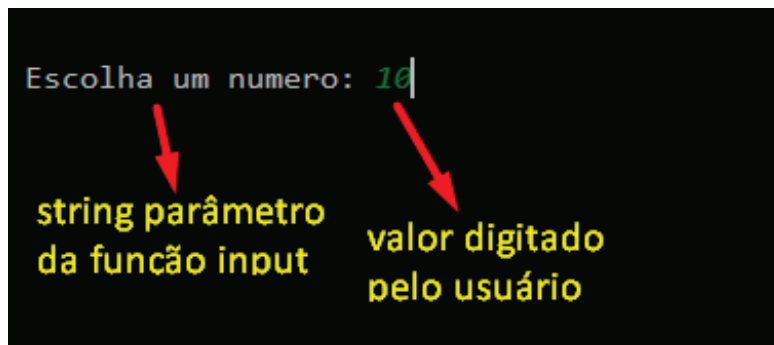
No código:

```
numero = input('Escolha um numero: ')
```



No terminal:

```
Escolha um numero: 10
```



Como vemos na imagem abaixo, o input sempre retorna um valor do tipo string (str), mesmo se for um numero inteiro ou um numero float

No código:

```
numero = input('Escolha um numero: ')  
print(type(numero))
```

No terminal:

```
Escolha um numero: 10  
<class 'str'>
```

Para corrigirmos isso podemos usar a função **int()** para transformar em um numero inteiro, ou **float()** para transformar em um numero real

```
A = int(input('Escolha um numero para A: '))  
print(type(A))  
  
B = float(input('Escolha um numero para B: '))  
print(type(B))
```

```
Escolha um numero para A: 5  
<class 'int'>  
  
Escolha um numero para A: 5.5  
<class 'float'>
```

STRINGS

O QUE É STRING?

É uma sequência de um ou mais caracteres. Para denominar uma string, em Python podemos utilizar aspas simples ('...') ou aspas duplas ("...")

CONCATENAÇÃO

Para concatenar duas ou mais strings, é utilizado o operador +.

No código:

```
a = 'império'
b = 'do'
c = "programador"

print(a + " " + b + " " + c)
```

↓ Espaço ↓ Espaço

No terminal:

```
      a      b      c
império do programador
```

↓ Espaço ↓ Espaço

MANIPULAÇÃO

Em python, existe algumas funções/métodos para ajudar na manipulação de strings. Serão listados abaixo os principais métodos

len()

Retorna o tamanho da string

```
a = 'império do programador'
print(len(a))
```

22

capitalize()

Retorna a string com a inicial maiúscula.

Atenção: Existem algumas funções que irão após a string separadas por um ponto, como no exemplo abaixo

```
a = 'império do programador'
a = a.capitalize()

print(a)
```

Império do programador

count()

Retorna quantas vezes um caractere (ou sequência de caracteres) aparece em uma string

```
a = 'império do programador'

print(a.count('o'))
```

4

startswith()

Retorna *True* ou *False*. Verifica se uma string inicia com uma sequência de caracteres ou não

```
a = 'império do programador'

print(a.startswith('imp'))
```

True

endswith()

Retorna *True* ou *False*. Verifica se uma string termina com uma sequência de caracteres ou não

```
a = 'império do programador'

print(a.endswith('imp'))
```

False

isalnum()

Retorna *True* ou *False*. Verifica se uma string contém letras e/ou números

```
a = '#$%*'

print(a.isalnum())
```

False

isalpha()

Retorna *True* ou *False*. Verifica se uma string contém apenas letras

```
a = 'ImpérioDoProgramador'
print(a.isalpha())
```

True

islower()

Retorna *True* ou *False*. Verifica se uma string está toda em minúscula

```
a = 'império do programador'
print(a.islower())
```

True

isupper()

Retorna *True* ou *False*. Verifica se uma string está toda em maiúscula

```
a = 'IMPÉRIO DO PROGRAMADOR'
print(a.isupper())
```

True

lower()

Transforma uma string em apenas caracteres minúsculos

```
a = 'IMPÉRIO DO PROGRAMADOR'
print(a.lower())
```

império do programador

upper()

Transforma uma string em apenas caracteres maiúsculos

```
a = 'império do programador'
print(a.upper())
```

IMPÉRIO DO PROGRAMADOR

swapcase()

Inverte o conteúdo da string (maiúsculo/minúsculo)

```
a = 'Império do Programador'
print(a.swapcase())
```

iMPÉRIO DO pROGRAMADOR

title()

Transforma a inicial de todas as letras em maiúsculo

```
a = 'império do programador'
print(a.title())
```

Império Do Programador

split()

Transforma uma string em uma lista, utilizando como referência os espaços (padrão)

```
a = 'império do programador'
print(a.split())
```

['império', 'do', 'programador']

replace(s1, s2)

Substitui na string, a parte s1 pela s2

```
a = 'império do programador'
print(a.replace('programador', 'josé'))
```

império do josé

find()

Retorna o índice da primeira ocorrência de um caractere. Se não haver, retorna -1

```
a = 'império do programador'
print(a.find('r'))
print(a.find('y'))
```

4
-1

ATENÇÃO: A INDEXAÇÃO
PYTHON COMEÇA EM ZERO

EX: i m p e r i o
 ↓ ↓ ↓ ↓ ↓ ↓ ↓
 0 1 2 3 4 5 6

ljust()

Ajusta a string num tamanho mínimo. Se necessário, adiciona espaços a direita

```
a = 'império do programador'
print(a.ljust(30))
```

império do programador

rjust()

Ajusta a string num tamanho mínimo. Se necessário, adiciona espaços a esquerda

```
a = 'império do programador'
print(a.rjust(30))
```

império do programador

center()

Ajusta a string num tamanho mínimo. Se necessário, adiciona espaços a esquerda e a direita

```
a = 'império do programador'
print(a.center(30))
```

império do programador

lstrip()

Remove todos os espaços em branco do lado esquerdo

```
a = '    império do programador    '
print(a.lstrip())
```

império do programador

rstrip()

Remove todos os espaços em branco do lado direito

```
a = '    império do programador    '
print(a.rstrip())
```

império do programador

strip()

Remove todos os espaços em branco à esquerda e à direita

```
a = '    império do programador    '
print(a.strip())
```

império do programador

FATIAMENTO

Com as fatias, podemos chamar vários valores de caracteres criando um intervalo de números de índice separados por dois pontos [x : y]:

Ao construir uma fatia, o primeiro número de índice é onde a fatia começa (incluso), e o segundo número do índice é onde a fatia termina (não incluso)

```
>>> a = 'império do programador'
>>> a[0:5]    de 0 (incluso) a 5 (não incluso)
'impér'
>>> a[:8]     até o índice 8 (não incluso)
'império '
>>> a[5:]     a partir do índice 5 (incluso)
'io do programador'
```

indexação da string

i	m	p	é	r	i	o		d	o		p	r	o	g	r	a	m	a	d	o	r
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21

NÚMEROS

OPERADORES ARITMÉTICOS

Retorna o resultado da operação

+	SOMA	<pre>>>> 5+2 7</pre>
-	SUBTRAÇÃO	<pre>>>> 5-2 3</pre>
*	MULTIPLICAÇÃO	<pre>>>> 5*2 10</pre>
/	DIVISÃO	<pre>>>> 10/2 5.0</pre>
%	RESTO DE DIVISÃO	<pre>>>> 10%3 1</pre>
**	POTÊNCIA	<pre>>>> 2**3 8</pre>

OPERADORES DE COMPARAÇÃO

Retorna *True* ou *False*

<	MENOR	<pre>>>> 5<10 True</pre>
<=	MENOR IGUAL	<pre>>>> 5<=5 True</pre>
>	MAIOR	<pre>>>> 10>3 True</pre>
>=	MAIOR IGUAL	<pre>>>> 10>=3 True</pre>
==	IGUAL	<pre>>>> 5==4 False</pre>
!=	DIFERENTE	<pre>>>> 5!=4 True</pre>

OPERADORES LÓGICOS

Retorna *True* ou *False*

NOT	INVERTE A CONDIÇÃO	<pre>>>> not True False</pre>
AND	TODAS AS CONDIÇÕES PRECISAM SER VERDADEIRA	<pre>>>> 1>0 and 0>1 False</pre>
OR	PRECISA DE APENAS UMA CONDIÇÃO VERDADEIRA	<pre>>>> 1>0 or 0<1 True</pre>

COLEÇÕES

LISTA

O QUE É LISTA?

Uma lista é uma estrutura de dados em Python que é uma sequência ordenada de elementos mutável. Cada elemento ou valor que está dentro de uma lista é chamado de item. Assim como as strings são definidas como caracteres entre aspas, as listas são definidas por ter valores entre colchetes []

Em python, as listas são declaradas da seguinte forma

Nome_da_Lista = [valor1, valor2, valor3, ..., valorN]

As listas podem conter qualquer tipo de valor: inteiro, float, string, inclusive outras listas, tuplas e dicionários que veremos mais a frente.

```
>>> lista = [1, 5.5, 'Império do programador', [1, 2, 3]]
>>> print(lista[2])
Império do programador
>>> print(lista[3])
[1, 2, 3]
>>> print(lista[3][1])
2
```

Para alterar um item da lista, você pode alterar usando o numero do indice, como na imagem abaixo

```
>>> lista = [1, 2, 3, 4, 5, 6]
>>> lista[3] = 'Morango'
>>> lista
[1, 2, 3, 'Morango', 5, 6]
```

MANIPULAÇÃO

Agora iremos ver sobre funções utilizadas para manipular uma lista

len()

Retorna o tamanho da lista

```
>>> numeros = [2, 5, 3, 7, 10, 6]
>>> len(numeros)
6
```

min()

Retorna o valor mínimo da lista

```
>>> numeros = [2, 5, 3, 7, 10, 6]
>>> min(numeros)
2
```

max()

Retorna o valor máximo da lista

```
>>> numeros = [2, 5, 3, 7, 10, 6]
>>> max(numeros)
10
```

sum()

Retorna o valor da soma de todos os elementos

```
>>> numeros = [2, 5, 3, 7, 10, 6]
>>> sum(numeros)
33
```

append()

Insere um valor no final da lista

```
>>> numeros = [1, 2, 3]
>>> numeros.append(4)
>>> numeros
[1, 2, 3, 4]
```

extend()

Insere uma lista no final de uma lista

```
>>> numeros = [1, 2, 3]
>>> numeros.extend([4, 5, 6])
>>> numeros
[1, 2, 3, 4, 5, 6]
```

del

Remove um elemento da lista, dado seu índice

```
>>> numeros = [1, 2, 3, 4, 5, 6]
>>> del numeros[4]
>>> numeros
[1, 2, 3, 4, 6]
```

in

Verifica se um número está na lista

```
>>> numeros = [1, 2, 3, 4, 5, 6]
>>> 3 in numeros
True
```

sort()

Ordena em ordem crescente

```
>>> numeros = [3,5,1,2,6,4]
>>> numeros.sort()
>>> numeros
[1, 2, 3, 4, 5, 6]
```

reverse()

Inverte a ordem dos elementos

```
>>> numeros = [1, 2, 3, 4, 5, 6]
>>> numeros.reverse()
>>> numeros
[6, 5, 4, 3, 2, 1]
```

OPERAÇÕES

Concatenação

Para juntar duas ou mais listas, podemos utilizar o operador +

```
>>> a = [1,2,3]
>>> b = [1,2,3]
>>> a+b
[1, 2, 3, 1, 2, 3]
```

Repetição

Para repetir uma lista, podemos utilizar o operador *

```
>>> lista = [1,2,3]
>>> lista*3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

FATIAMENTO

Os fatiamentos da lista ocorrem similares ao fatiamento de strings

```
>>> #[x:y] Entre X e Y-1
>>> lista = [1, 2, 3, 4, 5, 6]
>>> lista[2:5]
[3, 4, 5]
```

```
>>> #[x:] De X até o fim da lista
>>> lista = [1, 2, 3, 4, 5, 6]
>>> lista[2:]
[3, 4, 5, 6]
```

```
>>> #[x:y] Entre 0 e Y-1
>>> lista = [1, 2, 3, 4, 5, 6]
>>> lista[:4]
[1, 2, 3, 4]
```

CRIAÇÃO COM RANGE

Range() é uma função que define um intervalo de números inteiros. Utilizando junto à função list(), é possível criar uma lista com os valores de intervalo.

A função range pode receber 3 parâmetros

range(n)

Gera um intervalo de de 0 a n-1

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

range(i , n)

Gera um intervalo de de i a n-1

```
>>> list(range(5,10))
[5, 6, 7, 8, 9]
```

range(i , n , p)

Gera um intervalo de de i a n-1 com passo p

```
>>> list(range(2,20,2))
[2, 4, 6, 8, 10, 12, 14, 16, 18]
```

TUPLA

O QUE É TUPLA?

Uma tupla é uma estrutura de dados em Python que é uma sequência ordenada de elementos, assim como uma lista, porém imutável. Assim como as listas são definidas por ter valores entre colchetes [], as tuplas são definidas por terem valores entre parênteses

Em python, as tuplas são declaradas da seguinte forma

Nome_da_TUpla = (valor1, valor2, valor3, ..., valorN)

As listas podem conter qualquer tipo de valor: inteiro, float, string, inclusive outras tuplas, listas e dicionários.

```
>>> tupla = (1, 2, 3)
>>> tupla
(1, 2, 3)
>>> tupla[1]
2
```

```
>>> tupla = (1, [1,2,3], (5, 6, 7))
>>> tupla[1]
[1, 2, 3]
>>> tupla[1][1]
2
```

Não é possível alterar uma tupla

```
>>> tupla = (1, 2, 3)
>>> tupla[1] = 5
Traceback (most recent call last):
  File "<input>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

Uma utilização de tupla é o desempacotamento, que permite atribuir os elementos armazenados em uma tupla a diversas variáveis.

```
>>> tupla = (1, 2, 3)
>>> a, b, c = tupla
>>> a
1
>>> b
2
>>> c
3
```

DICIONÁRIO

O QUE É DICIONÁRIO?

Dicionários são usados para armazenar valores de dados em pares **chave : valor**.

Um dicionário é uma coleção não ordenada, mutável e não permite duplicatas.

Os dicionários são escritos com chaves { } e têm chaves e valores:

```
Nome_do_dicionário : {  
    "chave1" : valor1,  
    "chave2" : valor2,  
    "chave3" : valor3,  
    ...  
    "chaveN" : valorN;  
}
```

Exemplos:

Forma 1:

```
produtos = {  
    "chinelos": 49.90,  
    "camisa": 20.00,  
    "boné": 25.00,  
    "calça": 60  
}  
print(produtos)
```

```
{'chinelos': 49.9, 'camisa': 20.0, 'boné': 25.0, 'calça': 60}
```

Forma 2:

```
produtos = dict(chinelos=49.90, camisa=20.00, boné=25.00, calça=60)  
print(produtos)
```

```
{'chinelos': 49.9, 'camisa': 20.0, 'boné': 25.0, 'calça': 60}
```

OPERAÇÕES

Exemplos:

del

Exclui um item do dicionário

```
>>> produtos = dict(chinelos=49.90, camisa=20.00, boné=25.00, calça=60)  
>>> del produtos["chinelos"]  
>>> produtos  
{'camisa': 20.0, 'boné': 25.0, 'calça': 60}
```

keys()

Obtém as chaves de um dicionário

```
>>> produtos = dict(chinelos=49.90, camisa=20.00, boné=25.00, calça=60)  
>>> produtos.keys()  
dict_keys(['chinelos', 'camisa', 'boné', 'calça'])
```

values()

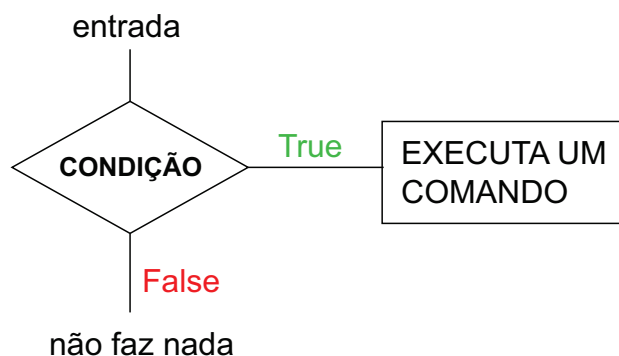
Obtém os valores de um dicionário

```
>>> produtos = dict(chinelos=49.90, camisa=20.00, boné=25.00, calça=60)  
>>> produtos.values()  
dict_values([49.9, 20.0, 25.0, 60])
```

ESTRUTURAS DE DECISÃO

ESTRUTURA IF

LÓGICA



SINTAXE

```
if <condição> :  
    <bloco de comandos>
```

EXEMPLO

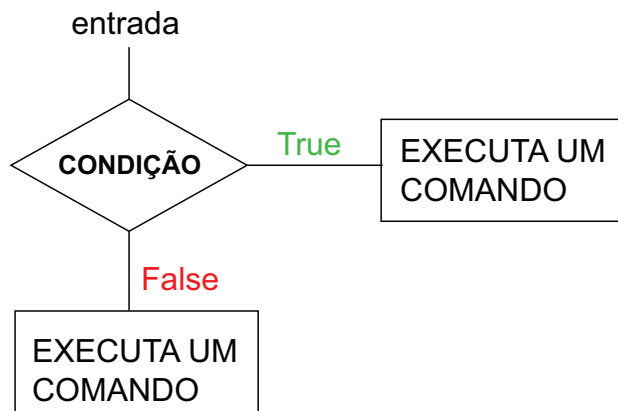
```
>>> valor = 10  
>>> if valor > 5:  
...     valor = valor * 3  
...  
>>> valor  
30
```

Para entendermos o que acontece aqui, vamos observar o seguinte: temos a variável **valor** que é igual a 10. Na linha seguinte temos a **condição para o if**: o valor é maior que 5? Sim, portanto retorna TRUE e a linha de baixo é executada

Como podemos ver, o valor que era 10 foi multiplicado por 3, agora a variável valor é igual a 30.

ESTRUTURA IF...ELSE

LÓGICA



SINTAXE

```
if <condição> :  
    <bloco de comandos>  
else:  
    <bloco de comandos>
```

EXEMPLO

Exemplo 1

```
>>> valor = 10  
>>> if valor>5:  
...     valor = 0  
...else:  
...     valor = 1  
...  
>>> valor  
0
```

Exemplo 2

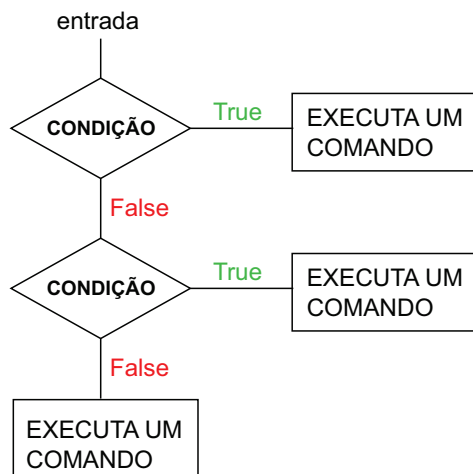
```
>>> valor = 3  
...if valor>5:  
...     valor = 0  
...else:  
...     valor = 1  
...  
>>> valor  
1
```

No primeiro caso, vemos que o valor (10) é maior que 5, logo retorna TRUE e o valor 0 é atribuído à variável **valor**

No segundo caso, o valor (3) não é maior que 5, logo retorna FALSE e o valor 1 é atribuído à variável **valor**

ESTRUTURA IF...ELIF...ELSE

LÓGICA



SINTAXE

```
if <condição> :  
    <bloco de comandos>  
elif:  
    <bloco de comandos>  
...  
elif:  
    <bloco de comandos>  
else:  
    <bloco de comandos>
```

Você pode colocar quantos **elif** quiser

EXEMPLO

```
>>> valor = 3  
>>> if valor == 1:  
...     print("zebra")  
...elif valor == 2:  
...     print("macaco")  
...elif valor == 3:  
...     print("cachorro")  
...elif valor == 4:  
...     print("gato")  
...else:  
...     print("Nenhum Animal")  
...  
cachorro
```

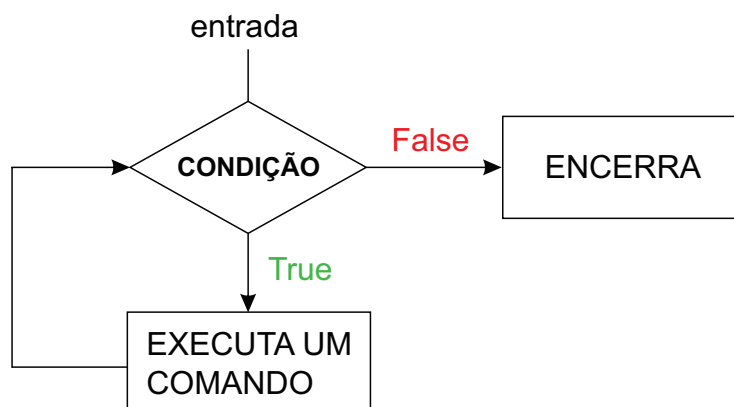
Como vemos no exemplo ao lado, ele vai testando cada condição de uma vez...

o valor é 1? NÃO
o valor é 2? NÃO
o valor 3 3? SIM
então imprime na tela “cachorro”
sai da estrutura if

ESTRUTURAS DE REPETIÇÃO

WHILE

LÓGICA



SINTAXE

```
while <condição> :  
    <bloco de comandos>
```

EXEMPLO

```
>>> a = 0  
>>> while a<10:  
...     print(a)  
...     a += 1 #Equevialente à (a = a + 1)  
...
```

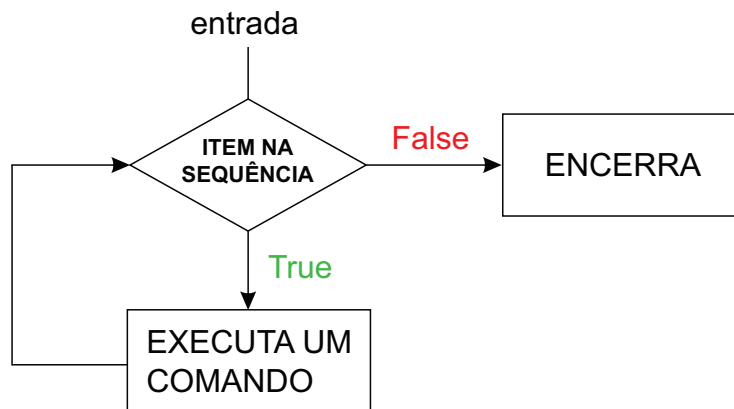
```
0  
1  
2  
3  
4  
5  
6  
7  
8  
9
```

Para entendermos o que acontece aqui, vamos observar o seguinte:

Inicialmente temos uma variável **a** que tem valor 0, após isso entraremos no laço **while** e irá verificar se a condição (**a < 10**). Se for verdadeiro ele executa o comando. No caso **a** era menor que 10, logo ele imprime na tela o valor de **a** e acrescenta 1, e repete esse ciclo até chegar em 9. Quando chega em 10 ele lê a condição e retorna **false**, portanto o laço while se encerra.

FOR

LÓGICA



SINTAXE

Um loop for é usado para iterar sobre uma sequência numérica, utilizando a função `range()` ou sobre listas, tuplas, dicionários, e strings.

for x in range() :
 <bloco de comandos>

for x in [lista] :
 <bloco de comandos>

EXEMPLO

```
>>> for x in range(5):  
...     print(x)  
...  
0  
1  
2  
3  
4
```

```
>>> lista = ['Joao','Gabriel', 'Lucia', 'Marcos']  
>>> for nome in lista:  
...     print(nome)  
...  
Joao  
Gabriel  
Lucia  
Marcos
```

```
>>> numeros = [1, 2, 3, 4, 5, 6]  
>>> soma = 0  
>>> #SOMAR TODOS OS NUMEROS DE UMA LISTA  
>>> for x in numeros:  
...     soma = soma + x  
...  
>>> soma  
21
```

FUNÇÕES

O QUE É FUNÇÃO?

Uma função é um bloco de código que só é executado quando é chamado. Você pode passar dados, conhecidos como parâmetros, para uma função. Uma função pode retornar dados como resultado. Funções são muito utilizadas para evitar escrever os mesmos algoritmos várias vezes.

SINTAXE

```
def nome_da_função ( <parâmetros> ) :  
    <bloco de comandos>
```

EXEMPLO

```
>>> def de_boas_vindas():  
...     print("Olá, seja bem vindo")  
...  
>>> de_boas_vindas()  
Olá, seja bem vindo
```

PARÂMETROS E ARGUMENTOS

Os termos parâmetro e argumento podem ser usados para a mesma coisa: informações que são passadas para uma função. Um **parâmetro** é a variável listada entre parênteses na definição da função. Um **argumento** é o valor enviado para a função quando ela é chamada.

```
>>> def soma(x, y):  
...     return x + y  
...  
>>> resultado = soma(2, 5)  
>>> resultado  
7
```

Diagrama de anotação:

- Dois pontos vermelhos apontam para `x` e `y` na linha `def soma(x, y):`, com o rótulo "parâmetro" à direita.
- Dois pontos vermelhos apontam para `2` e `5` na linha `soma(2, 5)`, com o rótulo "argumento" à direita.

ESCOPO DAS VARIÁVEIS

Variáveis que são definidas dentro de um corpo de função têm um **escopo local** e aquelas definidas fora têm um **escopo global**. Isso significa que as variáveis locais podem ser acessadas apenas dentro da função na qual são declaradas, enquanto as variáveis globais podem ser acessadas em todo o corpo do programa por todas as funções.

Exemplo com variável local:

Como vemos, não conseguimos acessar a variável *resultado* fora da função

```
>>> def soma(x, y):
...     resultado = x + y
...
>>> soma(2,3)
>>> resultado
Traceback (most recent call last):
  File "<input>", line 1, in <module>
NameError: name 'resultado' is not defined
```

Exemplo com variável global:

Declarando a variável como sendo global, conseguimos acessar a variável *resultado* fora da função

```
>>> def soma(x, y):
...     global resultado
...     resultado = x + y
...
>>> soma(2, 3)
>>> resultado
5
```

RETORNO DE VALORES

A instrução de retorno do Python é uma instrução especial que você pode usar dentro de uma função para enviar o resultado da função de volta ao usuário. Uma instrução de retorno consiste na palavra-chave **return** seguida por um valor de retorno opcional. O valor de retorno de uma função Python pode ser qualquer objeto Python.

```
>>> def multiplica(a, b):
...     return a * b
... x = multiplica(2, 5)
>>> x
10
```

BIBLIOGRAFIA

INSTALAÇÃO DO PYTHON. Disponível em: <https://www.devmedia.com.br/instalacao-do-python/40643>. Acesso em: 07 jan. 2021.

O que é Python, por que todos devem aprender e vantagens. Disponível em: <https://harve.com.br/blog/programacao-python-blog/o-que-e-python-vantagens-e-por-que-todos-devem-aprender/>. Acesso em: 07 jan. 2021.

INSTALAÇÃO Linux. Disponível em: <https://python.org.br/instalacao-linux/>. Acesso em: 07 jan. 2021.

HOW to install pycharm on mac. Disponível em: <https://www.educademy.co.uk/how-to-install-python-and-pycharm-on-mac>. Acesso em: 07 jan. 2021.

Como criar variáveis em Python. Disponível em: <https://blog.ffelix.eti.br/como-criar-variaveis-em-python/>. Acesso em: 08 jan. 2021.

PYTHON Dictionaries. Disponível em: https://www.w3schools.com/python/python_dictionaries.asp. Acesso em: 09 jan. 2021.